CSA0607-Design and Analysis of Algorithms

Topic: Dynamic Programming(Lab Expeiments)

Student Name: A. Sai Joshitha(192424220)

1. TRAVELLING SALESMAN PROBLEM

Aim

To find the **minimum cost tour** for the Travelling Salesman Problem (TSP) using the **Dynamic Programming** technique, where a salesman must visit every city exactly once and return to the starting city.

Algorithm

1. Start the program.
2. Read the number of cities.
3. Read the cost matrix representing the travelling cost between every pair of cities.
4. Consider city 0 as the starting city.
5. Represent the set of visited cities using a **bitmask**.
6. Define a DP function TSP(mask, current):
    - mask represents the cities already visited.
    - current represents the current city.
7. If all cities have been visited, return the cost of travelling back to the starting city.
8. For every unvisited city:
    - Visit the city.
    - Calculate the travelling cost.
    - Recursively find the minimum cost for the remaining cities.
9. Store already calculated states using memoization.

10. Select the minimum of all possible tours.

11. Display the minimum tour cost.

12. Stop.

Code

```
# Travelling Salesman Problem using Dynamic Programming
from functools import lru_cache # Read number of cities n =
int(input("Enter number of cities: "))
# Read cost matrix cost = []
print("Enter the cost matrix:") for i
in range(n):     row = list(map(int,
input().split()))     cost.append(row)
# Dynamic Programming function
@lru_cache(None) def tsp(mask,
current):     # If all cities are
visited     if mask == (1 << n) - 1:
return cost[current][0]
minimum = float('inf')     # Try
every unvisited city     for city in
range(n):

        if not (mask & (1 << city)):
            new_mask = mask | (1 << city)
current_cost = (
cost[current][city]
                + tsp(new_mask, city)
```

)

minimum = min(minimum, current_cost)

return minimum # Start from city 0 answer =

tsp(1, 0) print("Minimum tour cost:", answer)

Output

```
Enter number of cities: 5
Enter the cost matrix:
1 2 3 4 5
5 4 2 1 4
40 10 25 10 11
17 13 12 11 10
1 5 4 7 19
Minimum tour cost: 25
```

Time Complexity

$O(n^2 \times 2^n)$

Space Complexity

$O(n \times 2^n)$

Result

Thus, the **Travelling Salesman Problem was successfully solved using Dynamic Programming**, and the minimum cost tour was obtained.

2. ASSIGNMENT PROBLEM

Aim

To assign n jobs to n workers such that **each worker gets exactly one job** and the **total assignment cost is minimum**, using Dynamic Programming.

Algorithm

1. Start the program.
2. Read the number of workers and jobs.

3. Read the cost matrix.
4. Represent already assigned jobs using a bitmask. 5. Define a DP function assignment(worker, mask).
6. If all workers have been assigned jobs, return 0.
7. For the current worker, consider every unassigned job.
8. Calculate the cost of assigning each available job.
9. Recursively calculate the minimum cost for the remaining workers.
10. Store previously calculated states using memoization.
11. Select the assignment having the minimum total cost.
12. Display the minimum assignment cost.
13. Stop.

Code

```
# Assignment Problem using Dynamic Programming
from functools import lru_cache # Read number of
workers/jobs n = int(input("Enter number of
workers/jobs: "))
# Read cost matrix
cost = [] print("Enter the cost
matrix:") for i in range(n):     row =
list(map(int, input().split()))
cost.append(row)
# Dynamic Programming function
@lru_cache(None) def
```

```
assignment(worker, mask):     # All

workers are assigned     if worker ==

n:          return 0     minimum =

float('inf')     # Try every unassigned

job     for job in range(n):          if not

(mask & (1 << job)):

new_mask = mask | (1 << job)

current_cost = (

cost[worker][job]

                + assignment(worker + 1, new_mask)

            )

            minimum = min(minimum, current_cost)

return minimum # Start assignment from worker

0 answer = assignment(0, 0)


print("Minimum assignment cost:", answer)
```

Output

```
===== RESTART: C:/Users/Bhavya/AppDat
Enter number of workers/jobs: 4
Enter the cost matrix:
9 5 7 4
1 2 4 6
2 4 7 5
1 9 7 8
Minimum assignment cost: 13
|
```

Time Complexity

$O(n$^2×2^$n)$

Space Complexity

$O(2$^$n)$

Result

Thus, the Assignment Problem was successfully solved using Dynamic Programming, and the minimum total assignment cost was obtained.

3. 0/1 KNAPSACK

Aim

To determine the **maximum total value** that can be obtained by selecting items within a given knapsack capacity using the **0/1 Knapsack Dynamic Programming** technique.

Algorithm

1. Start the program.
2. Read the number of items.
3. Read the weight of each item.
4. Read the value of each item.
5. Read the capacity of the knapsack.
6. Create a DP table dp.
7. For every item and every possible capacity:
    - Check whether the item can be included.
8. If the item's weight is less than or equal to the current capacity:
    - Calculate the value when the item is included.
    - Calculate the value when the item is excluded.
    - Store the maximum of the two.
9. If the item cannot be included, copy the previous value.
10. The last cell of the DP table contains the maximum value.
11. Display the maximum value.
12. Stop.

Code

```
# 0/1 Knapsack using Dynamic Programming def

knapsack(weights, values, capacity):

  n = len(weights)

# Create DP table

dp = [

    [0] * (capacity + 1)

for _ in range(n + 1)

  ]

  # Fill the DP table    for i in

range(1, n + 1):        for w in

range(capacity + 1):           # If

item can be included           if

weights[i - 1] <= w:

include = (                    values[i -

1]

                + dp[i - 1][w - weights[i - 1]]

              )

              exclude = dp[i - 1][w]

dp[i][w] = max(include, exclude)

          else:

              dp[i][w] = dp[i - 1][w]    return

dp[n][capacity] # Read input n =

int(input("Enter number of items: "))
```

weights = list(map(int, input("Enter

weights: ").split())

) values = list(map(int, input("Enter

values: ").split())

)

capacity = int(input("Enter

knapsack capacity: ")

)

Calculate maximum value answer

= knapsack(weights, values,

capacity

)

print("Maximum value:", answer) **Output:**

```
Enter number of items: 4
Enter weights: 1 2 4 5
Enter values: 10 12 14 15
Enter knapsack capacity: 5
Maximum value: 24
```

Time Complexity

$O(nW)$

Space Complexity

$O(nW)$

Result

Thus, the **0/1 Knapsack Problem was successfully solved using Dynamic Programming**, and the maximum possible value within the given capacity was obtained.

4. LIST OPERATIONS

Aim

To implement basic list operations such as insertion, deletion, searching, and displaying elements using Python**.**

Algorithm

1. Start.
2. Read the elements of the list.
3. Display the original list.
4. Read the position and value for insertion.
5. Insert the value at the specified position.
6. Read the position for deletion.
7. Delete the element from the specified position.
8. Read an element for searching**.**
9. Search for the element in the list.
10. Display the final list and search result.
11. Stop.

Code

List Operations lst = list(map(int, input("Enter list

elements: ").split())) print("Original List:", lst) #

Insertion position = int(input("Enter position for

insertion: ")) value = int(input("Enter value to insert:

")) lst.insert(position - 1, value) print("List after

insertion:", lst)

Deletion position = int(input("Enter position for

deletion: ")) if 1 <= position <= len(lst):

deleted = lst.pop(position - 1)

print("Deleted element:", deleted)

print("List after deletion:", lst) else:

print("Invalid position")

Searching value = int(input("Enter element

to search: ")) if value in lst:

print("Element found at position:", lst.index(value) + 1)

else: print("Element not found")

Output

```
Enter list elements: 3
Original List: [3]
Enter position for insertion: 2
Enter value to insert: 10
List after insertion: [3, 10]
Enter position for deletion: 1
Deleted element: 3
List after deletion: [10]
Enter element to search: 3
Element not found
```

Time Complexity

- Insertion: O(n)
- Deletion: O(n)

- Searching: O(n)

Space Complexity

O(n)

Result

Thus, the basic list operations such as insertion, deletion, searching, and displaying elements were successfully implemented.

5.Floyd's Warshall Algorithm – Shortest Path

1. Aim

To implement **Floyd's Warshall algorithm** in Python to find the **shortest paths between all pairs of vertices** in a weighted graph.

2. Algorithm

1. Start with a weighted adjacency matrix representing the graph.
2. If there is no direct edge between two vertices, represent it using infinity (INF).
3. Set the distance matrix equal to the adjacency matrix.
4. Consider each vertex as an intermediate vertex one by one.
5. For every pair of vertices i and j, check whether going through vertex k gives a shorter path.
6. Update the distance using:

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

7. Repeat this process for all vertices.
8. The final matrix contains the shortest distance between every pair of vertices.

3. Python Code

```
# Floyd's Warshall Algorithm

# All Pairs Shortest Path
```

```
INF = 99999

def floyd_warshall(graph, n):

    # Create a copy of the graph

    dist = [row[:] for row in graph]

    # Consider each vertex as an intermediate vertex

    for k in range(n):

        for i in range(n):

            for j in range(n):

                if dist[i][k] + dist[k][j] < dist[i][j]:

                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

# Input

n = int(input("Enter the number of vertices: "))

print("Enter the adjacency matrix:")

print("Enter 99999 if there is no direct path.")

graph = []

for i in range(n):

    row = list(map(int, input().split()))

    graph.append(row)

# Find shortest paths

result = floyd_warshall(graph, n)

# Output

print("\nShortest Path Matrix:")
```

```
for i in range(n):
    for j in range(n):
        if result[i][j] == INF:
            print("INF", end=" ")
        else:
            print(result[i][j], end=" ")
    print()
```

Input

Use the following sample input:

Enter the number of vertices: 4

Enter the adjacency matrix:

0 5 99999 10

99999 0 3 99999

99999 99999 0 1

99999 99999 99999 0

The graph can be represented as:

From	To	Weight
1	2	5
1	4	10
2	3	3
3	4	1

Output

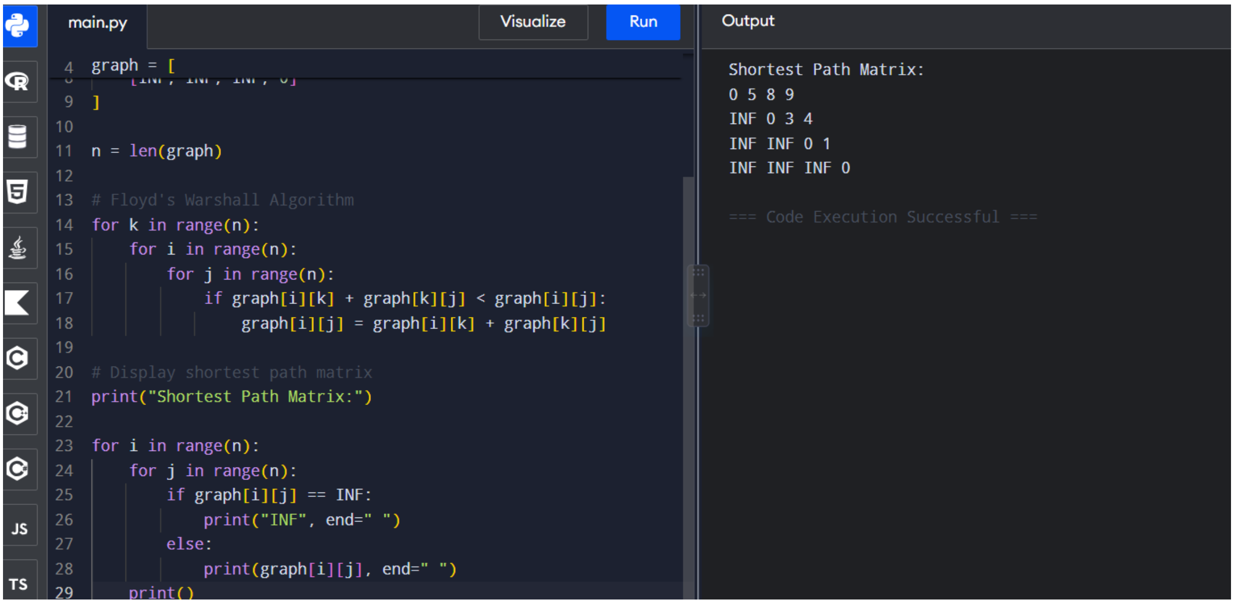
Shortest Path Matrix:

0 5 8 9

INF 0 3 4

INF INF 0 1

INF INF INF 0



Explanation of Output

- Shortest path from $\mathbf{1 \rightarrow 2 = 5}$
- Shortest path from $\mathbf{1 \rightarrow 3 = 5 + 3 = 8}$
- Shortest path from $\mathbf{1 \rightarrow 4 = 5 + 3 + 1 = 9}$
- Shortest path from $\mathbf{2 \rightarrow 3 = 3}$
- Shortest path from $\mathbf{2 \rightarrow 4 = 3 + 1 = 4}$
- Shortest path from $\mathbf{3 \rightarrow 4 = 1}$
- INF means there is no path between those vertices.

Time Complexity

Time Complexity: $O(V^3)$

Space Complexity: $O(V^2)$

where V is the number of vertices.

Result

Thus, the **Floyd's Warshall algorithm** was successfully implemented in Python to find the **shortest paths between all pairs of vertices** in a weighted graph.

6.SELECTION SORT

Aim

To sort a given array in ascending order using the Selection Sort algorithm.

Algorithm

1. Start.
2. Read the array elements.
3. Assume the first unsorted element is the minimum.
4. Compare it with the remaining elements.
5. Find the smallest element.
6. Swap it with the first unsorted element.
7. Repeat until the array is completely sorted.
8. Display the sorted array.
9. Stop.

Code

```
# Selection Sort def
selection_sort(arr):
   n = len(arr)     for i
in range(n - 1):
min_index = i

      for j in range(i + 1, n):
if arr[j] < arr[min_index]:
```

min_index = j arr[i], arr[min_index]

= arr[min_index], arr[i] arr = list(map(int,

input("Enter elements: ").split())) print("Original

array:", arr) selection_sort(arr) print("Sorted

array:", arr)

Output

```
Enter elements: 5 4 6 11 2
Original array: [5, 4, 6, 11, 2]
Sorted array: [2, 4, 5, 6, 11]
```

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

O(1)

Result

Thus, the given array was successfully sorted using the Selection Sort algorithm.

7. INSERTION SORT

Aim

To sort a given array in ascending order using the Insertion Sort algorithm.

Algorithm

1. Start.
2. Read the array.
3. Consider the second element as the key.
4. Compare the key with the previous elements.
5. Shift elements greater than the key to the right.
6. Insert the key into its correct position.
7. Repeat for all elements.
8. Display the sorted array.
9. Stop.

Code

```
# Insertion Sort def
insertion_sort(arr):     for i
in range(1, len(arr)):
        key = arr[i]         j = i - 1
while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]            j -= 1
        arr[j + 1] = key arr = list(map(int, input("Enter
elements: ").split())) print("Original array:", arr)
insertion_sort(arr) print("Sorted array:", arr)
```

Output

```
Enter elements: 4 5 7 2 1
Original array: [4, 5, 7, 2, 1]
Sorted array: [1, 2, 4, 5, 7]
```

Time Complexity

- Best Case: O(n)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

O(1)

Result

Thus, the given array was successfully sorted using the Insertion Sort algorithm.

8.OPTIMIZED BUBBLE SORT

Aim

To sort a given array using Optimized Bubble Sort by reducing unnecessary comparisons.

Algorithm

1. Start.
2. Read the array elements.
3. Compare adjacent elements.
4. Swap them if they are in the wrong order.
5. Set a flag whenever a swap occurs.
6. If no swap occurs during a pass, terminate the algorithm.

7. Repeat until the array is sorted.
8. Display the sorted array.
9. Stop.

Code

```
# Optimized Bubble Sort def
optimized_bubble_sort(arr):
   n = len(arr)     for i in
range(n - 1):         swapped =
False         for j in range(n - i
- 1):             if arr[j] > arr[j +
1]:                 arr[j], arr[j + 1]
= (                     arr[j + 1],
arr[j]
             )
             swapped = True
if not swapped:
          break arr = list(map(int, input("Enter
elements: ").split())) print("Original array:", arr)
optimized_bubble_sort(arr) print("Sorted array:",
arr)
```

Output

```
Enter elements: 4 5 2 3 1
Original array: [4, 5, 2, 3, 1]
Sorted array: [1, 2, 3, 4, 5]
```

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

$O(1)$

Result

Thus, the given array was successfully sorted using Optimized Bubble Sort.

9.CLOSEST PAIR OF POINTS

Aim

To find the pair of points having the minimum Euclidean distance among a given set of points.

Algorithm

1. Start.
2. Read the number of points.
3. Read the coordinates of all points.
4. Consider every possible pair of points.
5. Calculate the Euclidean distance between each pair. 6. Compare each distance with the minimum distance.

7. Store the pair having the minimum distance.
8. Display the closest pair and minimum distance.
9. Stop.

Code

```
# Closest Pair of Points import math n =
int(input("Enter number of points: "))
points = [] for i in range(n):
    x, y = map(float, input(          f"Enter
coordinates of point {i + 1}: "
    ).split())     points.append((x,
y)) minimum_distance =
float('inf') closest_pair = None
for i in range(n):     for j in
range(i + 1, n):         x1, y1 =
points[i]         x2, y2 = points[j]
distance = math.sqrt(

            (x2 - x1) ** 2 +
            (y2 - y1) ** 2
        )
        if distance < minimum_distance:
minimum_distance = distance
closest_pair = (                 points[i],
points[j]
            )
```

print("Closest pair:", closest_pair) print(

"Minimum distance:",

round(minimum_distance, 2)

)

Output

```
Enter number of points: 5
Enter coordinates of point 1: 1 0
Enter coordinates of point 2: 2 0
Enter coordinates of point 3: 3 1
Enter coordinates of point 4: 1 4
Enter coordinates of point 5: 5 5
Closest pair: ((1.0, 0.0), (2.0, 0.0))
Minimum distance: 1.0
```

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

Result

Thus, the pair of points having the minimum Euclidean distance was successfully determined.

10.CONVEX HULL

Aim

To find the Convex Hull of a given set of two-dimensional points**.**

Algorithm

1. Start.
2. Read the number of points.

3. Read the coordinates of all points.
4. Sort the points.
5. Construct the lower hull.
6. Construct the upper hull.
7. Combine the lower and upper hulls.
8. Display the points belonging to the Convex Hull.
9. Stop.

Code

```
# Convex Hull

def cross(o, a, b):

    return (

        (a[0] - o[0]) * (b[1] - o[1])

        - (a[1] - o[1]) * (b[0] - o[0])

    )

def convex_hull(points):

points = sorted(set(points))

if len(points) <= 1:

return points     # Lower Hull

lower = []     for point in

points:         while (

len(lower) >= 2             and

cross(                 lower[-2],

lower[-1],                 point
```

```
            ) <= 0
):
            lower.pop()
lower.append(point)     #
Upper Hull     upper = []     for
point in reversed(points):
while (              len(upper) >= 2
and cross(

                upper[-2],
upper[-1],                  point
            ) <= 0
):
            upper.pop()
upper.append(point)     return lower[:-1]
+ upper[:-1] n = int(input("Enter number
of points: ")) points = [] for i in range(n):
x, y = map(
        int,
        input("Enter x and y coordinates: ").split()
    )
    points.append((x, y)) hull =
convex_hull(points)
print("Points in Convex Hull:")
for point in hull:     print(point)
```

Output

```
Enter number of points: 5
Enter x and y coordinates: 0 0
Enter x and y coordinates: 0 1
Enter x and y coordinates: 1 1
Enter x and y coordinates: 1 2
Enter x and y coordinates: 2 2
Points in Convex Hull:
(0, 0)
(2, 2)
(1, 2)
(0, 1)
```

Time Complexity

O(n log n)

Space Complexity

O(n)

Result

Thus, the Convex Hull of the given set of points was successfully obtained.

11.BUBBLE SORT

Aim

To sort a given array in ascending order using the Bubble Sort algorithm**.**

Algorithm

1. Start.
2. Read the array elements.
3. Compare adjacent elements.
4. Swap them if the first element is greater than the second.
5. Repeat the process for all passes.
6. Display the sorted array.

7. Stop.

Code # Bubble Sort def

```
bubble_sort(arr):     n =
len(arr)     for i in range(n -
1):         for j in range(n - i -
1):             if arr[j] > arr[j +
1]:                 arr[j], arr[j + 1]
= (                     arr[j + 1],
arr[j]
                )
arr = list(map(int, input("Enter elements: ").split()))
print("Original array:", arr) bubble_sort(arr)
print("Sorted array:", arr)
```

Output

```
Enter elements: 10 1 14 2 5
Original array: [10, 1, 14, 2, 5]
Sorted array: [1, 2, 5, 10, 14]
```

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

O(1)

Result

Thus, the given array was successfully sorted using the Bubble Sort algorithm.

12. FIND MAXIMUM ELEMENT

Aim

To find the maximum element in an array .

Algorithm

1. Start.
2. Read the array elements.
3. Divide the array into two halves.
4. Recursively find the maximum element in the left half.
5. Recursively find the maximum element in the right half.
6. Compare the two maximum elements.
7. Return the larger element.
8. Display the maximum element.
9. Stop.

Code

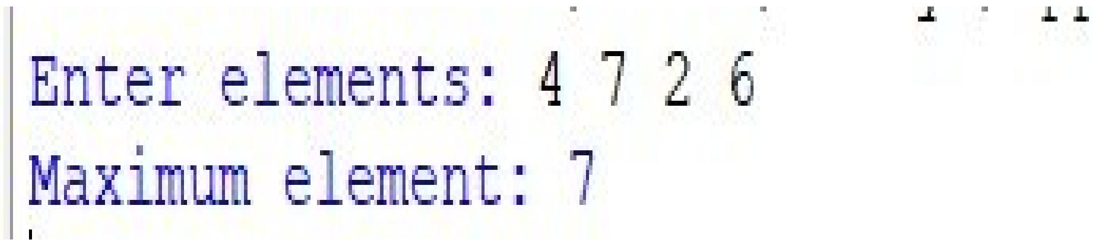
```
# Find Maximum using Divide and Conquer

def find_max(arr, left, right):

if left == right:
```

```
        return arr[left]
mid = (left + right) // 2
left_max = find_max(
        arr,
left,
mid
    )
    right_max = find_max(
        arr,
mid + 1,
right
    )
    return max(left_max, right_max) arr =
list(map(int, input("Enter elements: ").split()))
maximum = find_max(
    arr,     0,
len(arr) - 1
)
print("Maximum element:", maximum)
```

Output



Time Complexity

O(n)

Space Complexity

O(logn)

Result

Thus, the maximum element of the given array was successfully .

13. SORT AND FIND MAXIMUM

Aim

To sort a given array and find its maximum element using sorting**.**

Algorithm

1. Start.
2. Read the array elements.
3. Sort the array in ascending order.
4. The last element of the sorted array is the maximum.
5. Display the sorted array.
6. Display the maximum element.
7. Stop.

Code

```
# Sort and Find Maximum

arr = list(map(int, input("Enter elements: ").split()))

print("Original array:", arr) arr.sort() print("Sorted

array:", arr)


print("Maximum element:", arr[-1])
```

Output

```
Enter elements: 5 1 2 4 6
Original array: [5, 1, 2, 4, 6]
Sorted array: [1, 2, 4, 5, 6]
Maximum element: 6
```

Time Complexity

O(n log n)

Space Complexity

O(n)

Result

Thus, the given array was successfully sorted and its maximum element was obtained.

14. BINARY SEARCH

Aim

To search for a given element in a sorted array using the Binary Search algorithm.

Algorithm

1. Start.
2. Read the sorted array.
3. Read the element to be searched.
4. Set low = 0 and high = n - 1.
5. Calculate the middle position.
6. If the middle element equals the search element, return its position.
7. If the search element is smaller, search the left half.

8. Otherwise, search the right half.
9. Repeat until the element is found or the search range becomes empty.
10. Display the result.
11. Stop.

Code # Binary Search def

```
binary_search(arr, key):
   low = 0     high =
len(arr) - 1     while
low <= high:
      mid = (low + high) // 2
if arr[mid] == key:
         return mid
elif key < arr[mid]:
high = mid - 1
else:
         low = mid + 1
return -1 arr =
list(map(
   int,
   input("Enter sorted elements: ").split()
))
key = int(input("Enter element to search: ")) result
= binary_search(arr, key)

if result != -1:
```

print(

"Element found at position:",

result + 1

)

else: print("Element not

found")

Output

```
Enter sorted elements: 1 2 3 4
Enter element to search: 2
Element found at position: 2
```

Time Complexity

- Best Case: O(1)
- Average Case: O(log n)
- Worst Case: O(log n)

Space Complexity

O(1)

Result

Thus, the required element was successfully searched using Binary Search.

15. EFFICIENT SORTING — MERGE SORT

Aim

To efficiently sort a given array using the Merge Sort algorithm .

Algorithm

1. Start.
2. Read the array elements.
3. Divide the array into two halves.
4. Recursively sort the left half.
5. Recursively sort the right half.
6. Merge the two sorted halves.
7. Continue until the entire array is sorted.
8. Display the sorted array.
9. Stop.

Code # Merge Sort def

```
merge_sort(arr):     if len(arr)
<= 1:         return arr     mid =
len(arr) // 2     left =
merge_sort(arr[:mid])     right =
merge_sort(arr[mid:])     result
= []     i = 0

    j = 0

    while i < len(left) and j < len(right):
if left[i] <= right[j]:
result.append(left[i])             i += 1
else:
```

```
        result.append(right[j])
j += 1
result.extend(left[i:])
result.extend(right[j:])
return result arr = list(map(
   int,
   input("Enter elements: ").split()
)) print("Original array:", arr)
sorted_array = merge_sort(arr)
print("Sorted array:", sorted_array)
```

Output

```
Enter elements: 4 1 5 6
Original array: [4, 1, 5, 6]
Sorted array: [1, 4, 5, 6]
```

Time Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

Space Complexity

O(n)

Result

Thus, the given array was successfully sorted using the efficient Merge Sort algorithm.

16. STRING COMPARISON USING STACK

Aim

To compare two strings using the Stack data structure and determine whether they are equal.

Algorithm

1. Start.
2. Read the first string.
3. Read the second string.
4. Create two empty stacks.
5. Push each character of the first string into the first stack.
6. Push each character of the second string into the second stack.
7. Compare the top elements of both stacks.
8. If any characters differ, declare the strings unequal.
9. If all characters match, declare the strings equal.
10. Display the result.
11. Stop.

Code # String Comparison

using Stack def

compare_strings(s1, s2):

stack1 = [] stack2 = [] #

Push first string characters for

ch in s1: stack1.append(ch)

Push second string characters

for ch in s2:

stack2.append(ch) # Check

lengths if len(stack1) !=

len(stack2): return False #

Compare elements while

stack1: char1 = stack1.pop()

char2 = stack2.pop() if char1

!= char2: return False

return True s1 = input("Enter first

string: ") s2 = input("Enter second

string: ") if compare_strings(s1,

s2): print("Strings are equal")

else: print("Strings are not

equal")

Output

```
Enter first string: 11 12
Enter second string: 11 12
Strings are equal
```

Time Complexity

O(n)

Space Complexity

O(n)

Result

Thus, the two given strings were successfully compared using the Stack data structure.